LAPORAN PRAKTIKUM PBO

PEKAN 6



OLEH:

HAMDI SIDQI ALIFI

NIM 2511531009

PRAKTIKUM A

MATA KULIAH PRAKTIKUM

PEMOGRAMAN BERORIENTASI

OBJEK

DOSEN PENGAMPU : AFDHAL DINILHAK, M.KOM.

ASISTEN PRAKTIKUM : SITI FADHILAH RAHMI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

A. Identitas dan output tools.NimParams

Nama : Hamdi Sidqi Alifi
NIM : 2511531009
Kelas : DIF61118/IF/Prkt/A

```
as\abstraction-2511531009> java -cp bin
ools.NimParams 2511531009
Parameter tugas untuk NIM 2511531009
  CARD_ADMIN_PERCENT  = 1
  TRANSFER_ADMIN_FEE  = 4500
  SMS_MAX_LENGTH      = 109
```

```
public final class StudentInfo {
    public static final String NIM = "2511531009";    // TO
    public static final String NAME = "Hamdi Sidqi Alifi";

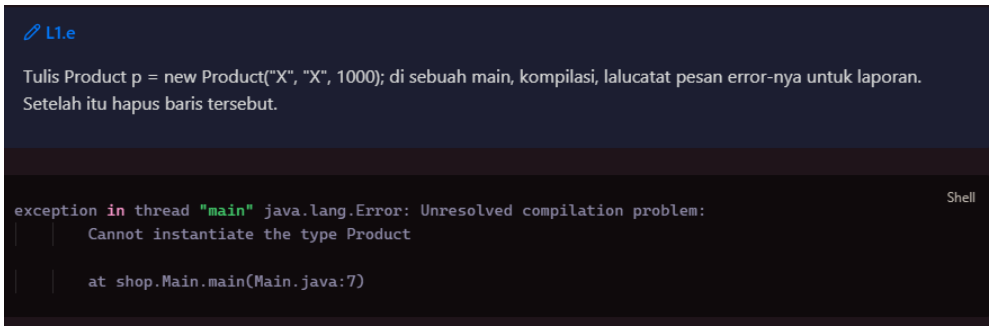
    public static final int CARD_ADMIN_PERCENT = 1;    // T
    public static final long TRANSFER_ADMIN_FEE = 4500;  //
    NimParams
    public static final int SMS_MAX_LENGTH = 109;        //
```

```
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511531009 | Hamdi Sidqi Alifi

-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (1%, Rp4500, 109 karakter)
```

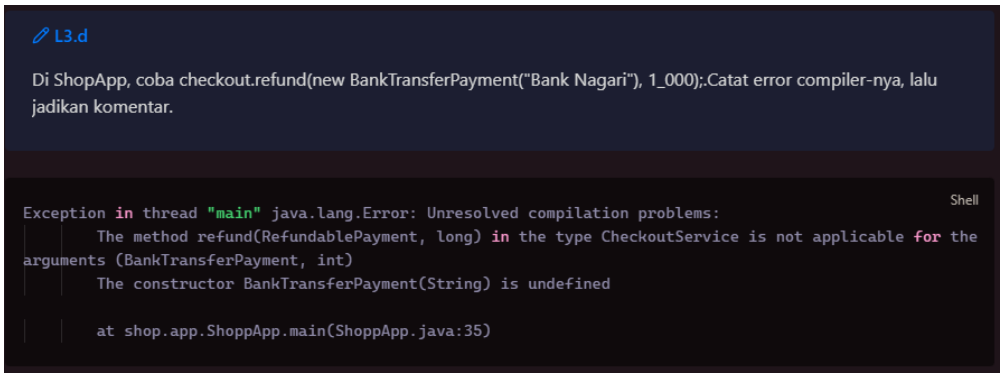
B. Tangkapan Layar Error Compiler pada L1.e, L3.d, L4.a, T1.a

i. L1.e



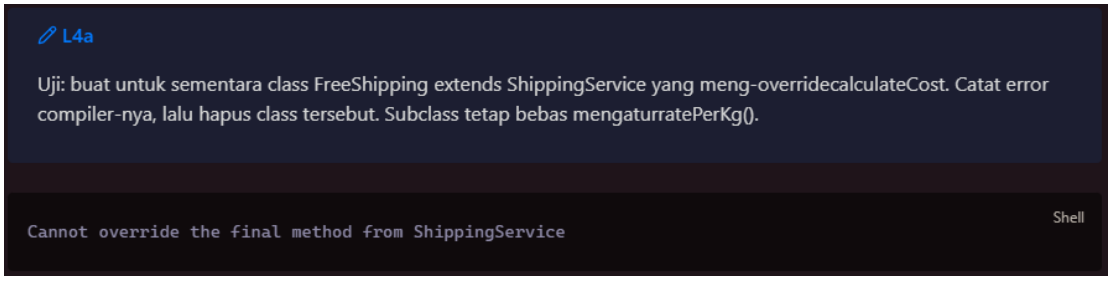
Error di atas dikarenakan compiler mendeteksi adanya sebuah abstract class yang di inisialisasikan pada class yang bukan merupakan tujuan inherinctancenya

ii. L3.d



Error tersebut terjadi karena BankTransferMethod tidak daapt mengaplikasikan method refund() dari checkoutservice

iii. L4.a

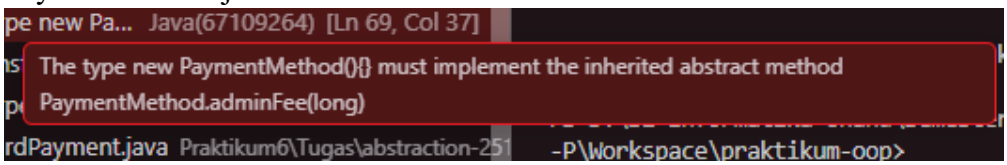


Error terjadi karena method berstatus final tidak dapat di override

iv. T1

a. Cara yang salah dari evolusi API dan konlik diamond

- Menambahkan method abstract long adminFee(long amount); ke PaymentMethod.java



Hal ini salah karena pada class yang memanggil PaymentMethod tidak memanggil method adminFee()

Class berikut juga terdampak error karena merupakan class yang mengimplementasikan paymentmethod

a. BankTransferPayment.java
b. CreditCardPayment.java
c. EwalletPayment.java

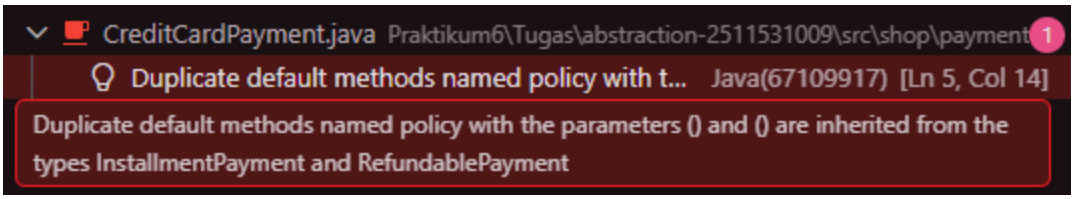
Solusi dari masalah ini adalah dengan mengubah abstarct menjadi default

```
default long adminFee(long
amount) {
    return 0;
};
```

- Anonymous class voucher juga ikut rusak karena voucher juga ikut memanggil PaymentMethod karena ia merupakan sebuah objct dari PaymentMethod

```
PaymentMethod voucher = new
PaymentMethod() {
    private long remaining =
    50_000;
```

v. T1.e



Uji coba T1.e menghasilkan error, karena class CreditCardPayment mengimplementasikan RefundablePayment dan InstallmentPayment yang sama sama memiliki method policy() yang beraksi secara berbeda. Hal ini membuat compiler mengonfirmasikan kembali untuk menunjukkan method mana yang harus digunakan,

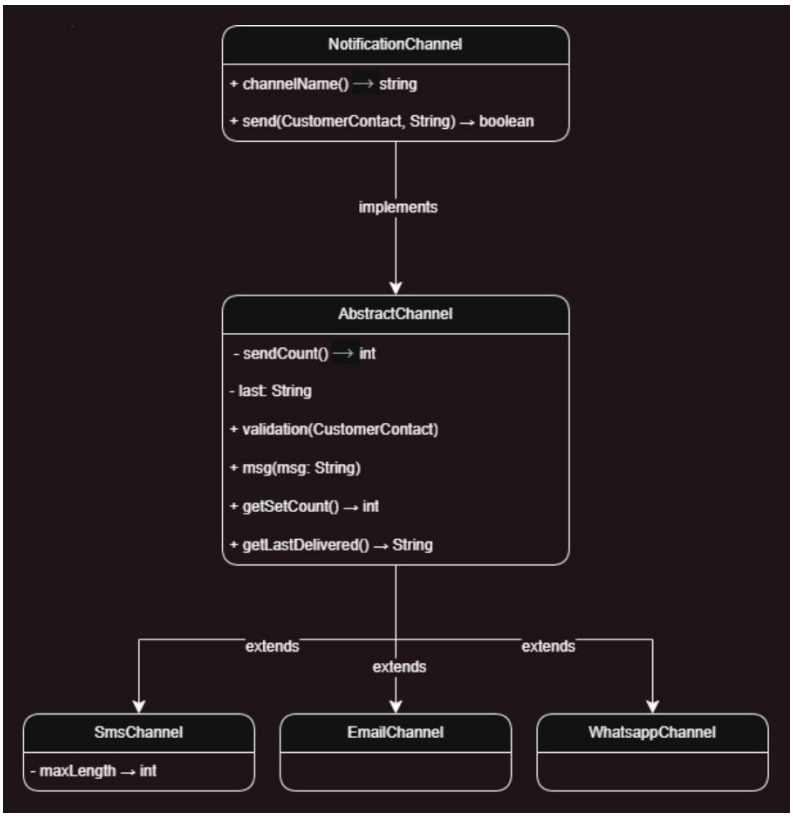
C. Jawaban pertanyaan tugas

i. T1

a. Pada shop.payment.EWalletPayment tidak perlu meng-override policy() karena hanya terdapat implementasi RefundablePayment, tidak ada implementasi Payment Method sehingga tidak ada tabrakan.

b. Java memilih salah satu default sacara otomatis karena untuk menghindari ambiguitas dan kesalahan dalam memberikan keluaran.

ii. Diagram UML T2



iii. T2

a. Mengapa ada interface dan abstract class, bukan salah satunya saja?
Class dapat terdiri dari kumpulan object lain dan class mewakili sesuatu yang punya identitas, perilaku, dan value.

Inteface fokus pada method sebagai kontrak tanpa adanya kode di dalamnya. Pada interface tidak disimpan sebuah state. Satu kelas boleh menerapkan banyak.

Method pada abstact class tidak kosong, state pada sebuah abstract class juga tidak kosong, dan sebuah class memiliki paling banyak satu superclass, karena abstract class juga merupakan class, untuk satu class hanya boleh memiliki paling banyak satu.

Jika tidak menggunakan interface maka layanan harus bergantung pada abstract class, bisa saja sebuah class tidak membutuhkan semuanya.

Penggunaan keduanya bertujaun untuk meghindari menulis ulang kode dia antara anggota yang telah menjadi subkelas dari sebuah abstract class.

b. Apa akibatnya jika send(...) tidak dibuat final?

Jika tidak ada field final pada send(…) maka bagaimana cara kita untuk mencegah send(…) dioverride pada class lain. Final mencegah terjadinya untuk sebuah method yang siudah bekerja sebagai mana mestinya.

c. Mengapa NotificationService menyimpan List<NotificationChannel>, bukan List<AbstractChannel>?
Hal ini dikarenakan NotificationService bekerja sebagai method

iv. T3

a. Temukan 5 masalah desain yang berkaitan dengan materi abstraction (abstract class, interface, default method, final, nested class)

1. Interface terlalu gemuk karena menggabungkan pay(), refund(), ship, dan printInvoice dalam satu interface.
2. Courier adalah abstract class tetapi method cost() sudah konkret dan mengandalakan switch
3. Method default apply() memeriksa this instanceof MemberDiscount dan this instanceof FlashSaleDiscount untuk menentukan diskon.
4. Address punya field non-final dan setter publik (setStreet, setCity). SimpleOrder menyimpan referensi langsung ke objek Address yang sama.
5. JneCourier mewarisi Courier tetapi tidak menambah atau meng-override apa pun.

```
Total cart: 170000
Ongkir EXPRES 2 kg: 0
ORD-1 -> Jl. Sudirman 10, Bukittinggi
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | Voucher, remaining balance 70000.0
Exception in thread "main" java.lang.UnsupportedOperationException: Voucher is not shipped
        at review.GiftVoucher.ship(BadShop.java:41)
        at review.BadShop.main(BadShop.java:184)
```

b. Tabel

No.	Lokasi	Masalah	Dampak Nyata	Perbaikan yang Diusulkan
1	GiftVoucher implements store operations	GiftVoucher dipaksa mengimplementasi refund() dan ship() yang tidak relevan untuk voucher, sehingga melempar UnsupportedOperationException.	Bug nyata	Pecah menjadi interface kecil.

2	JneCourier	Courieradalah abstract class tetapi methodcost()sudah konkret dan mengandalkanswitchpada string"REGULAR"/"EXPRESS"/"SAME_DAY". JneCourier extends Courier` tetapi tidak override apa pun — subclass tidak menambah perilaku. Abstraction menjadi ilus	Bug nyata	Jadikan cost() abstract; setiap subclass (RegularShipping, ExpressShipping, SameDayShipping) mengimplementasi tarifnya sendiri.
3	Dsicount	apply()memeriksathis instanceof MemberDiscountdanthis instanceof FlashSaleDiscount` untuk menentukan diskon. Ini anti-pattern: perilaku seharusnya didelegasikan ke masing-masing implementasi, bukan ditumpuk di interface.	Risiko	Buat apply() abstract (tanpa default) dan biarkan MemberDiscount.apply() mengembalikan price - price*5/100 serta FlashSaleDiscount.apply() mengembalikan price - price*30/100. Atau tambah method abstract percentage() yang di-override tiap class, lalu
4	Simple Order	Address punya field non-final dan setter publik (setStreet, setCity). SimpleOrder menyimpan referensi langsung ke objek Address yang sama. Ketika second.getAddress().setCity(" Bukittinggi") dipanggil, objek	Bug nyata	Buat apply() abstract (tanpa default) dan biarkan MemberDiscount.apply() mengembalikan price - price*5/100

		Address yang dipakai bersama ikut berubah.		serta FlashSaleDiscount.apply() mengembalikan price - price*30/100. Atau tambah method abstract percentage() yang di-override tiap class, lalu default apply() memakainya.
5	JneCourier extends Courier	JneCourier mewarisi Courier tetapi tidak menambah atau meng-override apa pun. Abstract classCourier` punya method konkret, jadi subclass tidak dipaksa berkontribusi. Polimorfisme tidak terjadi.	Risiko	Dengan memecahkan masalah #2 (method cost() abstract), JneCourier akan dipaksa mengimplementasi cost() sendiri, sehingga setiap subclass benar-benar punya perilaku berbeda.

c. Refactor minimal 3 masalah ke file baru src/review/fixed/BadShopRefactored.java (package review.fixed).

```
Total cart: 170000
Ongkir EXPRESS 2 kg: 36000
ORD-1 -> Jl. Sudirman 10, Padang
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | Voucher, remaining balance 70000
```

v. Tabel T4

a. Abstract Class vs Interface

Pertanyaan desain	Abstract class atau interface?	Alasan	Contoh dari proyek

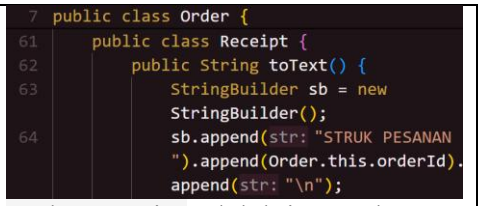
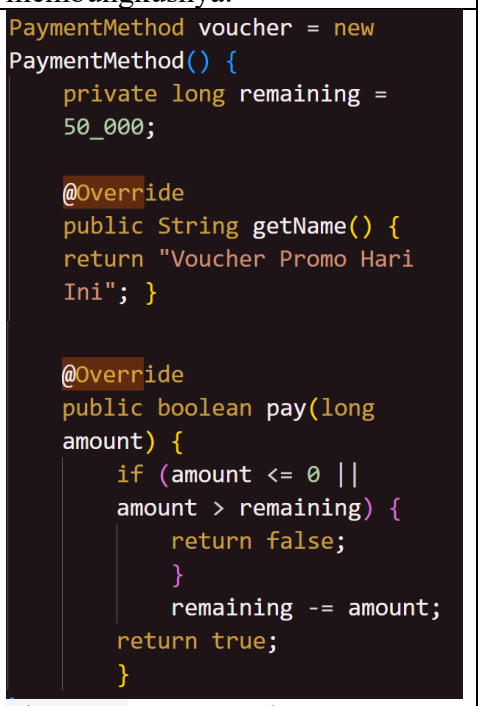
Subclass berbagi field dan konstruktor	**Abstract class**	Abstract class dapat mendeklarasikan field (instance state) dan konstruktor yang diwariskan ke subclass. Interface hanya bisa menyimpan konstanta static final, tidak ada instance field atau konstruktor.	`public PhysicalProduct(String sku, String name, long price, int weightGrams) { super(sku, name, price); if (weightGrams <= 0) { throw new IllegalArgumentException (s: "Weight must not be negative nor zero"); } this.weightGrams = weightGrams;`
			
			`public DigitalProduct(String sku, String name, long price, String downloadUrl) { super(sku, name, price); if (downloadUrl == null || downloadUrl.isBlank()) { throw new IllegalArgumentException (s: "Download Url must not be blank"); } this.downloadUrl = downloadUrl;`
			
			Product (abstract class) memiliki field sku, name, price dan konstruktor yang memvalidasi harga negatif. PhysicalProduct dan DigitalProduct cukup memanggil super(sku, name, price) lalu menambah field masing-masing (weightGrams / downloadUrl).

Satu class perlu beberapa kemampuan sekaligus	**Interface**	Java hanya mendukung single inheritance untuk class (extends satu class), tetapi satu class dapat implements banyak interface. Jika class butuh menggabungkan beberapa kemampuan, interface adalah satu-satunya pilihan.	`public class CreditCardPayment implements RefundablePayment, InstallmentPayment { private final String cardHolder; private final long creditLimit; private long usedCredit;` CreditCardPayment implements RefundablePayment **dan** InstallmentPayment sekaligus (diamond inheritance, T1.4). Kartu kredit bisa di-refund **dan** dicicil. Ini tidak mungkin dilakukan dengan abstract class karena Java

			tidak mendukung multiple class inheritance.
Rumus bersama yang tidak boleh diubah subclass	**Abstract class**	Abstract class dapat menandai method sebagai final sehingga subclass tidak boleh meng-override-nya. Interface tidak bisa memakai final pada instance method. Semua method interface (non-static, non-private) bersifat public dan dapat di-override.	`public final long calculateCost(int weightGrams) { if (weightGrams <= 0) { return 0; } long kilograms = (weightGrams + 999L) / 1000; return kilograms * ratePerKg(); }` ShippingService.calculateCost(int grams) adalah final method. Rumus ratePerKg() * (grams/500 + sisa) sama untuk semua kurir. Subclass hanya menyediakan ratePerKg() dan tidak boleh mengubah cara calculateCost bekerja.
Kontrak yang perlu ditambah method baru di kemudian hari	**Interface**	Interface dapat ditambah method baru dengan default (atau static/private) method tanpa merusak implementasi yang sudah ada. Jika abstract class ditambah abstract method baru, semua subclass yang ada harus mengimplementasinya atau gagal kompilasi.	`default long adminFee(long amount) { return 0; };` PaymentMethod ditambah adminFee(long) sebagai **default method** (T1.1). Implementasi lama seperti EWalletPayment tidak perlu diubah karena default method memberi implementasi bawaan (mengembalikan 0). Jika PaymentMethod adalah abstract class, penambahan abstract method adminFee akan memaksa semua subclass mengimplementasinya.

b. Jenis Nested Class

Kondisi	Static nested, inner, atau anonymous?	Alasan	Contoh dari proyek
Class pembantu tidak membaca data objek luar	**Static nested class**	Jika class pembantu tidak mengakses instance field outer class, static nested class menghindari referensi implisit ke outer object yang tidak perlu. Lebih ringan dan independen.	`class Cart { static class Item { private final String name; private final long price; private final int quantity;` Cart.Item di BadShopRefactored dideklarasikan static class Item karena Item tidak membaca field items atau state Cart lainnya.

Objek harus membaca field objek luarnya	**Inner class (non-static)**	Inner class membawa referensi implisit ke instance outer class, sehingga dapat langsung mengakses field dan method instance outer tanpa perlu referensi eksplisit.	 Order.Receipt adalah inner class non-static karena Receipt.toText() membaca order.getId(), order.getItemsTotal(), dan alamat tujuan dari instance Order. Jadi Receipt perlu akses ke state instance Order yang membungkusnya.
Implementasi sekali pakai, tidak perlu nama	**Anonymous class**	Jika implementasi interface hanya dipakai di satu tempat dan tidak akan dipanggil ulang, anonymous class menghindari pembuatan class bernama terpisah. Kode lebih ringkas.	 ShopApp menggunakan anonymous class yang mengimplementasi PaymentMethod. Contohnya metode pembayaran "tunai/COD" sekali pakai yang dibuat langsung di tempat pemakaian, tanpa perlu file class terpisah.

D. Tangkapan Layar `TugasCheck`, `SelfCheck`, keluaran `TugasApp`

a. TugasCheck

1. Checkpoint T1

```
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511531009 | Hamdi Sidqi Alifi

-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (1%, Rp4500, 109 karakter)

-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh,
Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan

=== HASIL: 5 / 10 pemeriksaan lulus ===
```

2. Checkpoint T2

```
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511531009 | Hamdi Sidqi Alifi

-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (1%, Rp4500, 109 karakter)

-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan

-- T2 - Sistem notifikasi
  [LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...) final
  [LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel

[SMS] to 081234567890 xxxx
```

```
=== SELF CHECK: Praktikum Abstraction ===


-- L1 - Abstract class & abstract method
  [LULUS] L1.1 Product adalah abstract class
  [LULUS] L1.2 Product punya abstract method category()
  [LULUS] L1.3 PhysicalProduct & DigitalProduct adalah subclass konkret Product
  [LULUS] L1.4 Konstruktor menolak harga negatif (IllegalArgumentException)
  [LULUS] L1.5 getInfo() PhysicalProduct sesuai format
  [LULUS] L1.6 ShippingService abstract dengan abstract method ratePerKg()
  [LULUS] L1.7 calculateCost(): Reguler 700g=10000, 1001g=20000, 0g=0; Ekspres 700g=18000

-- L2 - Interface & program to an interface
  [LULUS] L2.1 Shippable interface; PhysicalProduct implements, DigitalProduct tidak
  [LULUS] L2.2 PaymentMethod interface dengan getName() dan pay(long)
The amount must not be negative, zero nor exceed balance.
  [LULUS] L2.3 EWalletPayment: bayar sukses mengurangi saldo, saldo kurang ditolak
  [LULUS] L2.4 CheckoutService.checkout(...) menerima PaymentMethod & ShippingService (bukan class konkret)
  [LULUS] L2.5 Total keranjang contoh: Reguler=260000, Ekspres=268000

-- L3 - Default, static, private method; interface inheritance
  [LULUS] L3.1 Discountable: applyDiscount default, percentOf static, ada private method
  [LULUS] L3.2 DigitalProduct implements Discountable; harga akhir 50000 -> 45000
  [LULUS] L3.3 RefundablePayment & InstallmentPayment adalah interface turunan PaymentMethod
  [LULUS] L3.4 CreditCardPayment implements RefundablePayment DAN InstallmentPayment
  [LULUS] L3.5 monthlyInstallment(100000, 3)=33334 dan 13 bulan ditolak
  [LULUS] L3.6 CheckoutService.refund(...) hanya menerima RefundablePayment

-- L4 - final method & final class
  [LULUS] L4.1 ShippingService.calculateCost(int) adalah final method
  [LULUS] L4.2 Address final class, semua field private final, tanpa setter

-- L5 - Static nested, inner, dan anonymous class
  [LULUS] L5.1 Order.Line static nested; Order.Receipt inner (non-static)
  [LULUS] L5.2 Receipt.toText() memuat id pesanan, total barang, dan kota tujuan
  [LULUS] L5.3 ShopApp memakai anonymous class yang mengimplementasikan PaymentMethod

=== HASIL: 23 / 23 pemeriksaan lulus ===
```

c. TugasApp

```
Tugas Oleh Hamdi Sidqi Alifi (2511531009)
== T1: Total dengan biaya admin (Ekspres) ==
E-Wallet Dompetku: Rp268000 (admin Rp0)
Kartu Kredit (Siti Rahma): Rp270680 (admin Rp2680)
Transfer bank nagari: Rp272500 (admin Rp4500)

Kebijakan kartu kredit: Refund maksimal 7 hari; Cicilan maksimal 12
Kebijakan e-wallet: Refund maksimal 7 hari
== T2: Notifikasi Pesanan ==

[EMAIL] to siti@example.com Pesanan ORD-0005 sudah dikirim. Terima kasih telah berbelanja di toko UMKM kami!

[SMS] to 081234567890 Pesanan ORD-0005 sudah dikirim. Terima kasih telah berbelanja di toko UMKM kami!

[WA] to 081234567890 [Toko UMKM] Pesanan ORD-0005 sudah dikirim. Terima kasih telah berbelanja di toko UMKM kami!

[LOG] Siti Rahma: Pesanan ORD-0005 sudah dikirim. Terima kasih telah berbelanja di toko UMKM kami!
Terkirim ke Siti Rahma lewat 4 kanal


[WA] to +6281298765432 [Toko UMKM] Pesanan ORD-0005 sudah dikirim. Terima kasih telah berbelanja di toko UMKM kami!

[LOG] Hendra: Pesanan ORD-0005 sudah dikirim. Terima kasih telah berbelanja di toko UMKM kami!
Terkirim ke Hendra lewat 2 kanal
```

E. Link Praktikum6/Tugas pada Repo:
https://github.com/HamdiSAlifi/PrakPBO_A_2511531009/tree/after-uts/Praktikum6/Tugas/abstraction-2511531009